

Scriptorium Parser Benchmark: A Reproducible Ingestion Pipeline for Research-Focused RAG

Krish Sarkar

University of California, Merced
krishsarkar@ucmerced.edu

Abstract

Document parsing is a foundational component of retrieval-augmented generation (RAG) systems, yet it is often treated as a minor preprocessing step. In research-focused systems such as Scriptorium / RefAIned, parser choice affects chunking, embedding quality, retrieval coherence, citation traceability, and the preservation of scholarly document structure. This report extends an earlier retrieval-oriented parser evaluation into a reproducible CSE 195 ingestion benchmark.

We implement a modular parser framework that standardizes execution across nine local PDF parsers: PyMuPDF, pdfplumber, PyPDF, Apache Tika, Unstructured, Docling, Marker, MinerU, and GROBID. The framework uses a shared parser registry, individual parser wrappers, a common `extract(pdf_path)` interface, normalized text outputs, artifact tracking, and benchmark status reporting. On an IEEE-style scientific PDF, all nine parsers successfully execute in the stabilized JupyterHub environment. The benchmark records runtime, extracted character count, output file inventory, and qualitative output previews.

The results show a clear tradeoff between speed and structural richness. Flat text parsers such as PyMuPDF, PyPDF, and pdfplumber run quickly but provide limited structural preservation. Structure-aware parsers such as Docling, Marker, and MinerU produce richer Markdown, layout, image, and artifact outputs at higher runtime cost. GROBID extracts less total text but provides academically useful metadata-oriented structure. The benchmark also documents excluded parsers, including Nougat and PaddleOCR, whose dependency or runtime constraints made them unsuitable for the stable shared environment. Overall, this work provides a reproducible ingestion baseline for future retrieval, chunking, citation, and traceability experiments in Scriptorium. This report focuses on parser execution, output normalization, artifact tracking, and infrastructure readi-

ness rather than final retrieval-quality evaluation.

1 Introduction

Large Language Models (LLMs) have become central to modern NLP systems (Vaswani et al., 2017; Devlin et al., 2019), especially in retrieval-augmented generation (RAG) pipelines where external documents are used to improve factual grounding and contextual relevance. In these systems, document ingestion is not a neutral preprocessing step. The way a PDF is parsed determines what text, metadata, headings, tables, figures, and citations are available to downstream chunking and retrieval components.

Scientific PDFs are particularly challenging because they often contain multi-column layouts, page headers and footers, section hierarchies, equations, figures, tables, and references. Flat text extraction can lose this structure, merge unrelated regions, or introduce spacing artifacts. These errors can then propagate into chunks, embeddings, search results, and generated answers. Document parsing research therefore emphasizes that useful parsing should preserve structural relationships, semantic organization, and machine-readable representations such as Markdown, JSON, or XML rather than merely transcribing text (Zhang et al., 2024).

This CSE 195 project focuses on Scriptorium / RefAIned, a research-oriented ingestion and retrieval system. The goal is to move beyond ad hoc parser testing and create a reproducible benchmark that can support evidence-backed parser selection. This report updates the earlier CSE 188 parser evaluation by shifting the emphasis from a small retrieval experiment to a reusable ingestion infrastructure. Instead of only asking which parser gives better retrieved chunks, this work asks how a practical system can consistently run, compare, and inspect multiple parsers under a shared framework.

The contributions of this report are as follows:

- We implement a modular parser benchmark with a shared registry, standardized parser wrappers, normalized outputs, and parser status reporting.
- We successfully run nine local parsers on the same scientific PDF: PyMuPDF, pdfplumber, PyPDF, Tika, Unstructured, Docling, Marker, MinerU, and GROBID.
- We compare parser outputs by extracted character count, runtime, artifact production, and qualitative output previews.
- We document excluded tools and environment constraints, including Nougat, PaddleOCR, Uni-Parser / Uniparser, and PyMuPDF4LLM.
- We identify next steps for retrieval-focused evaluation, including chunking comparisons, top- k retrieval tests, and structure preservation metrics.

2 Background and Motivation

2.1 Document Parsing for RAG

RAG systems typically include document loading, parsing, chunking, embedding, retrieval, and generation. Many systems focus on the embedding model or retrieval algorithm, but parser output quality strongly shapes what information is available before embedding begins. If a parser loses headings, splits tables incorrectly, duplicates headers, or merges columns, those artifacts become part of the retrieval corpus.

Document parsing can be viewed as a spectrum. At one end are flat text extractors that recover text quickly with limited structural awareness. In the middle are semi-structured extractors that produce readable text but still preserve limited document hierarchy. At the other end are structure-aware parsers that attempt to preserve sections, tables, figures, metadata, layout information, and artifacts. This distinction matters for research PDFs because the most useful context is often tied to structure, not just raw text.

2.2 From CSE 188 Evaluation to CSE 195 Infrastructure

The earlier CSE 188 paper evaluated how parser choice affected chunk coherence and retrieval usefulness. It found that similarity scores alone did not

fully capture output quality: PyMuPDF achieved strong cosine similarity, while Docling produced higher human-rated coherence. This motivated a deeper CSE 195 follow-up focused on infrastructure. The current project builds a reusable benchmark that can run multiple parsers under one interface, save outputs, compare runtime and coverage, and support future retrieval experiments.

3 System Design

3.1 Parser Framework

The benchmark is organized around a refactored parser framework. The framework includes:

- `parsers/core/base.py`: shared parser result types and PDF validation.
- `parsers/core/registry.py`: parser registration and dependency metadata.
- `parsers/core/runner.py`: benchmark execution and output writing.
- `parsers/implementations/`: individual parser wrappers.

Each parser exposes a common `extract(pdf_path)` function and returns a standardized `ParserResult`. This design allows the benchmark runner to execute different tools consistently even though their underlying APIs and outputs differ.

3.2 Output Organization

Each parser writes a normalized `.txt` output into: `outputs/slam_llm/parser_outputs/`

Structure-aware parsers may also create parser-specific artifact folders. In the final benchmark, these include:

- `docling_run/`
- `marker_run/`
- `mineru_run_txt/`

These folders preserve additional Markdown files, JSON metadata, extracted images, layout PDFs, and intermediate artifacts. This is important because a RAG system may eventually need more than plain text: it may need tables, captions, figures, references, and source traceability.

Parser	Category	Role in Benchmark
PyMuPDF	Flat text	Fast baseline extraction
pdfplumber	Flat text	Layout-sensitive text baseline
PyPDF	Flat text	General PDF text baseline
Tika	Semi-structured	Java-based document extraction
Unstructured	Structure-aware	Element-labeled text extraction
Docling	Structure-aware	Markdown-oriented structured parsing
Marker	Structure-aware	Markdown and artifact extraction
MinerU	Structure-aware	Layout-aware parsing with artifacts
GROBID	Structured metadata	Scholarly document and metadata extraction

Table 1: The nine parsers included in the final stable benchmark.

3.3 GROBID Service Requirement

Most parsers run directly inside the Python environment, but GROBID requires a running service. Because Docker was unavailable in the JupyterHub environment, GROBID 0.9.0 was built from source and launched with Java 21 using:

```
cd /home/jovyan/grobid-0.9.0
export JAVA_HOME=/opt/conda
export PATH=$JAVA_HOME/bin:$PATH
./gradlew :grobid-service:run
```

The benchmark verifies the service using `http://localhost:8070/api/isalive` before running the parser.

4 Benchmark Setup

4.1 Input Document

The benchmark uses the SLAM-LLM scientific PDF as the test document. This document is a realistic academic-paper input with dense technical content, multi-column layout, headings, authors, abstract, figures, tables, and references.

4.2 Parsers Evaluated

The final stable benchmark evaluates nine parsers. These include established PDF/text extraction tools, document extraction frameworks, and modern structure-aware parsers, including PyMuPDF, pdfplumber, PyPDF, Apache Tika, Unstructured, Docling, Marker, MinerU, and GROBID (PyMuPDF, 2026; jsvine, 2026; PyPDF, 2026; Apache Software Foundation, 2026; Unstructured, 2026; Docling Project, 2026; Datalab, 2026; OpenDataLab, 2026; GROBID, 2026). Table 1 groups them by extraction category.

5 Results

5.1 Benchmark Status

All nine selected parsers successfully ran on the same input PDF. Table 2 summarizes the final status, extracted character count, and runtime.

Parser	Characters	Runtime (s)	Type
PyMuPDF	88,199	0.09	Flat text
pdfplumber	85,465	1.74	Flat text
PyPDF	88,394	0.54	Flat text
Tika	90,660	0.09	Semi-structured
Unstructured	95,836	4.56	Structure-aware
Docling	100,840	10.34	Structure-aware
Marker	102,133	40.50	Structure-aware
MinerU	88,200	31.92	Structure-aware
GROBID	69,846	2.02	Structured metadata

Table 2: Final 9/9 parser benchmark results on the SLAM-LLM PDF.

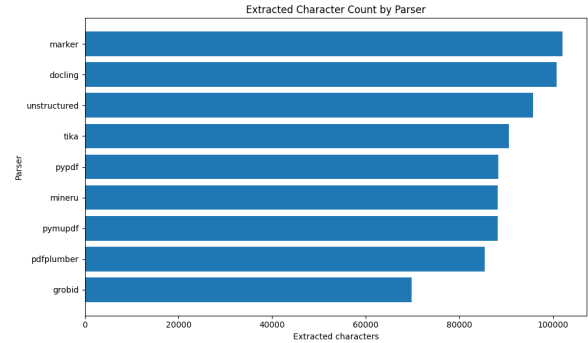


Figure 1: Extracted character count by parser. Structure-aware parsers generally produced more text, while GROBID produced less text because of its scholarly-structure focus.

5.2 Character Count Comparison

Character count provides a rough indication of extraction coverage, but it is not a direct quality metric. Higher character count may indicate more complete extraction, but it may also include duplicated headers, noisy OCR output, references, captions, or formatting artifacts. In the final run, Marker produced the most text, followed by Docling and Unstructured. GROBID produced the least text, which is expected because it emphasizes scholarly document structure and metadata rather than maximizing raw text volume.

5.3 Runtime Comparison

The runtime comparison shows a clear tradeoff between simple extraction and heavier structure-aware parsing. PyMuPDF, Tika, and PyPDF were the fastest. Marker and MinerU were the slowest because they perform more expensive document analysis and produce additional artifacts.

5.4 Output Inventory

The normalized output directory contains one .txt output per parser. Docling, Marker, and MinerU also produce artifact folders. In the final

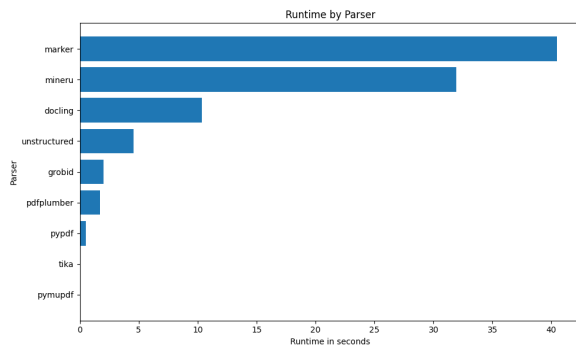


Figure 2: Runtime by parser. Structure-aware parsers generally require more time than flat text extractors because they perform layout analysis, model inference, OCR-related steps, or artifact generation.

run, docling_run contained one Markdown artifact, marker_run contained seven files including Markdown, metadata, and extracted images, and mineru_run_txt contained thirty files including Markdown, JSON, image assets, and layout PDFs.

This artifact inventory is important because the benchmark is not only measuring text extraction. It is also identifying which tools produce reusable structural outputs that could support later retrieval, citation, or multimodal processing.

6 Qualitative Findings

6.1 Flat Text Parsers

PyMuPDF, pdfplumber, and PyPDF are useful baselines because they are fast and simple. However, they provide limited structural preservation. They may lose heading hierarchy, merge text across columns, or flatten tables and figures. pdfplumber in particular can preserve some layout signals but is sensitive to multi-column formatting and spacing artifacts.

6.2 Tika

Tika provides a semi-structured baseline and runs quickly once Java is available. It produces readable text and can handle document-level extraction, but it does not preserve rich scholarly structure or artifact outputs in the same way as modern structure-aware parsers.

6.3 Unstructured

Unstructured provides element-level labels such as headers, narrative text, and plain text. This makes it more useful than flat extraction for downstream chunking because chunks can potentially be

aligned with document elements rather than arbitrary character ranges.

6.4 Docling

Docling provides a strong balance between structural output and runtime. It produced one of the highest character counts while remaining much faster than Marker and MinerU. Its Markdown-style output makes it useful for RAG pipelines that benefit from headings and readable structure.

6.5 Marker

Marker produced the largest normalized text output and generated additional artifacts. Its output preserved Markdown formatting and extracted images. The tradeoff is runtime: Marker was the slowest parser in the final benchmark. It is useful when richer structure and artifact extraction are more important than speed.

6.6 MinerU

MinerU required the most setup work but successfully ran using the txt method and pipeline backend. It produced Markdown, JSON, images, layout PDFs, and other artifacts. MinerU is valuable for layout-aware parsing and multimodal-ready ingestion, though it has higher runtime and environment complexity.

6.7 GROBID

GROBID produced fewer total characters than the other parsers, but it is valuable for academic metadata and scholarly document structure. It extracts title, authors, references, and structured academic text. Its main tradeoff is that it requires a running service at localhost:8070.

7 Excluded Parsers and Environment Constraints

7.1 Nougat

Nougat was tested and produced .mmd output, but it required older dependency pins, especially an older transformers version. That conflicted with the modern parser stack used by Docling, Marker, and MinerU. It was removed from the final benchmark to preserve a clean 9/9 stable run.

7.2 PaddleOCR

PaddleOCR was considered as an OCR/layout baseline. It installed and downloaded model files, and PDF pages were successfully rendered to images

with PyMuPDF. However, runtime inference failed in the JupyterHub environment due to a PaddlePaddle backend issue. Because it required separate environment isolation, it was excluded from the stable benchmark.

7.3 Uni-Parser / Uniparser

The originally linked `cjm-sfw/Uniparser` project was not a PDF parser. It focused on image/human parsing. A separate document-oriented Uni-Parser exists in the literature and is relevant as future work, but no simple local CLI or package was available for this benchmark.

7.4 PyMuPDF4LLM

PyMuPDF4LLM was considered but excluded because it overlaps heavily with PyMuPDF. Since the benchmark already includes PyMuPDF as a flat text baseline, adding PyMuPDF4LLM would not add a sufficiently distinct parser category.

8 Connection to Scriptorium Roadmap

This benchmark supports the broader Scriptorium roadmap by turning parser selection into an evidence-backed engineering decision. A research-focused RAG system needs more than raw extracted text. It needs reliable ingestion, metadata, source traceability, reproducibility, and clear trade-offs between runtime and structure preservation.

The parser benchmark establishes a foundation for future Scriptorium work in three ways. First, it provides a standardized framework for adding or removing parsers. Second, it saves normalized outputs and artifacts that can be reused for chunking and retrieval experiments. Third, it documents environment constraints so future development can decide when parser isolation is necessary.

9 Next Steps

The next stage is to move beyond extraction success and evaluate parser quality for retrieval. Suggested next steps include:

- Chunk each parser output using a consistent chunking strategy.
- Measure chunk counts and average chunk length across parsers.
- Run embedding-based retrieval over each parser output.

- Compare top- k retrieval quality using a fixed query set.
- Evaluate structure preservation, including headings, tables, captions, references, and figure text.
- Track runtime and artifact size as practical engineering tradeoffs.
- Explore environment isolation for dependency-sensitive parsers such as Nougat and PaddleOCR.

10 Conclusion

This project demonstrates that document parsing should be treated as a first-class component of research-focused RAG infrastructure. The final benchmark successfully runs nine local parsers under a standardized interface and records output coverage, runtime, artifacts, and qualitative differences. The results show that flat text parsers are fast and useful baselines, while structure-aware parsers provide richer outputs that are better aligned with downstream retrieval and traceability needs.

The benchmark also shows that parser selection is not only a quality question but an infrastructure question. Some tools require Java services, some require model inference, and some introduce dependency conflicts. By documenting these trade-offs and creating a reproducible benchmark, this work provides a practical foundation for future Scriptorium experiments in chunking, retrieval, citation grounding, and source-aware document intelligence.

References

- Apache Software Foundation. 2026. Apache tika documentation. <https://tika.apache.org/>. Accessed May 2026.
- Datalab. 2026. Marker: Convert documents to markdown, json, and html quickly. <https://github.com/datalab-to/marker>. Accessed May 2026.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. *NAACL*.
- Docling Project. 2026. Docling documentation. <https://docling-project.github.io/docling/>. Accessed May 2026.
- GROBID. 2026. Grobid documentation. <https://grobid.readthedocs.io/>. Accessed May 2026.

- jsvine. 2026. pdfplumber: Plumb a pdf for detailed information about each text character, rectangle, and line. <https://github.com/jsvine/pdfplumber>. Accessed May 2026.
- OpenDataLab. 2026. Mineru: Document extraction and conversion tool. <https://github.com/opedatalab/MinerU>. Accessed May 2026.
- PyMuPDF. 2026. Pymupdf documentation. <https://pymupdf.readthedocs.io/>. Accessed May 2026.
- PyPDF. 2026. Pypdf documentation. <https://pypdf.readthedocs.io/>. Accessed May 2026.
- Unstructured. 2026. Unstructured documentation. <https://docs.unstructured.io/>. Accessed May 2026.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, and 1 others. 2017. Attention is all you need. *NeurIPS*.
- Qintong Zhang, Bin Wang, Victor Shea-Jay Huang, Junyuan Zhang, Zhengren Wang, Hao Liang, Conghui He, and Wentao Zhang. 2024. Document parsing unveiled: Techniques, challenges, and prospects for structured information extraction. *arXiv preprint arXiv:2410.21169*.